

AGRUPAMENTO DE ESCOLAS EDUARDO GAGEIRO

PROVA DE APTIDÃO PROFISSIONAL • 2025/2026

Mage TD

Curso Profissional de Gestão e Programação de Sistemas Informáticos

Escola Secundária de Sacavém • Agrupamento de Escolas Eduardo Gageiro

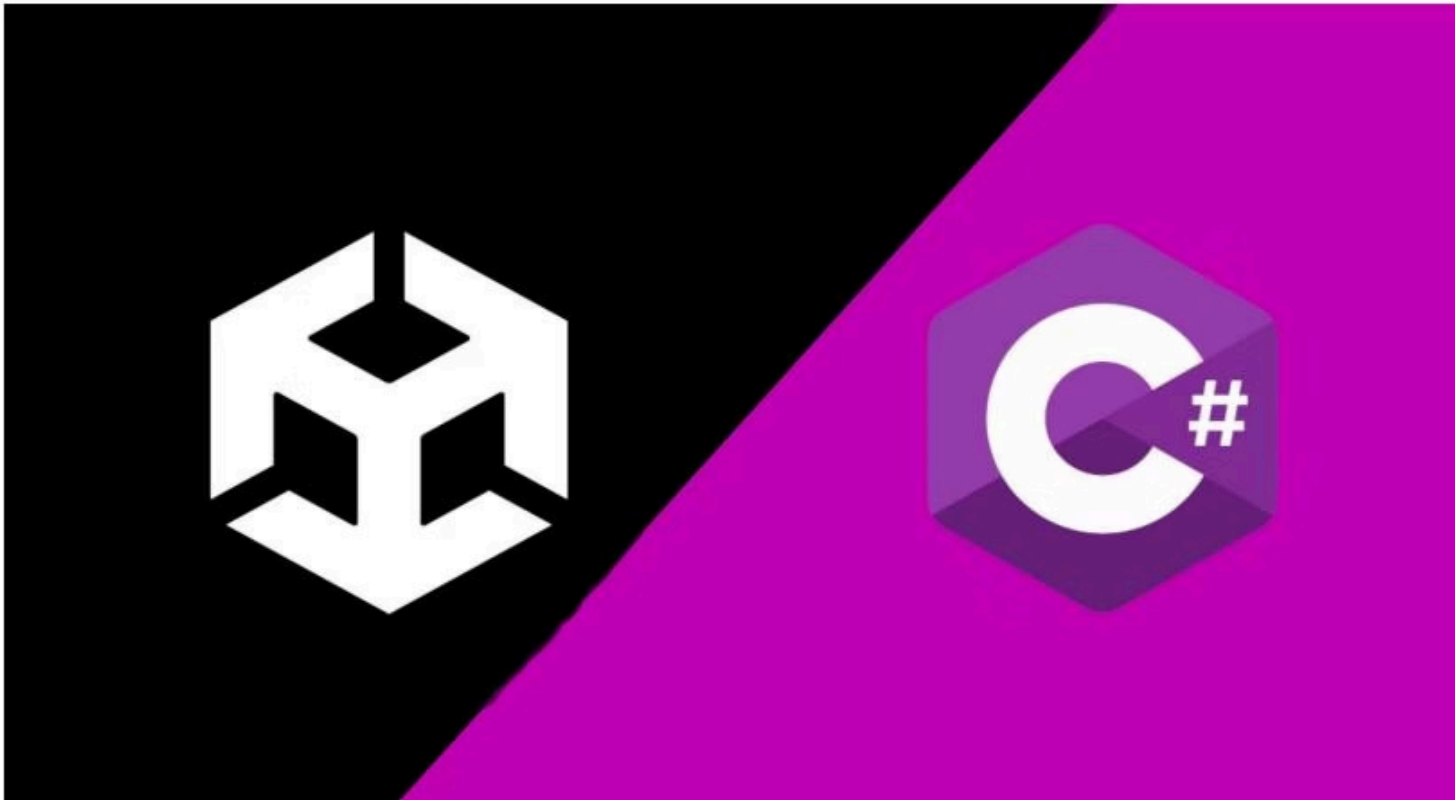
David Kouadio • Professoras Orientadoras: Cláudia Barata & Paula Alcobia



Introdução

O projeto consistiu no desenvolvimento de um jogo digital do género **First-Person Shooter (FPS)**, denominado **Mage TD**, no qual o jogador recorre a magos e feitiços para avançar na história.

O principal objetivo foi a aplicação prática dos conhecimentos adquiridos ao longo do curso, especialmente nas áreas de **programação** e **lógica computacional**, utilizando o motor **Unity** e a linguagem **C#**.



Tecnologias Envolvidas

Unity & C#

Motor de jogos e linguagem principal para cenários, mecânicas, lógica e exportação Web.

VS Code, Git & GitHub

IDE para o código-fonte e controlo de versões do projeto.

Blender & Assets

Modelos 3D, recursos gráficos e sonoros para enriquecer a experiência.

Unity Play & LibreOffice

Publicação da versão jogável online e produção do relatório.

Cronograma de Desenvolvimento

01

Planeamento

Definição do tipo de jogo, mecânicas principais, estilo visual e objetivos.

03

Mecânicas Principais

Sistema de inimigos, magos, ataques, pontos de vida e magia.

05

Testes e Correção

Testes funcionais, correção de falhas e apreciação de críticas.

02

Criação no Unity

Configuração inicial do ambiente e organização da estrutura do projeto.

04

Interface Gráfica

Criação de menus, HUD, indicadores de vida e progresso.

06

Publicação Beta

Exportação para a plataforma Web e publicação no Unity Play.

Aspetos Técnicos



Desenvolvimento Modular

O projeto foi desenvolvido de forma modular, recorrendo a scripts em C# para controlar os diferentes componentes do jogo.

- **Código-fonte:** Scripts de movimento, ataques, colisões e gestão de objetos.
- **Interface (UI):** Elementos visuais para vida, magia e estado da partida.

- **Lógica do jogo:** Controlo do estado do player e transições entre fases.
- **Cenas:** Organização clara das fases e navegação entre menus e níveis.



A História de Aerfran

Num mundo regido pela magia, o reino de **Aerfran** era assolado por invasões. Um lendário mago ergueu uma muralha com **4 torres**, defendendo o reino sozinho.

Com o tempo, a magia foi integrada no reino. Após o falecimento do mago, um jovem casal de magos salvou Aerfran de uma grande invasão e recebeu título de nobreza. Os seus **4 filhos** tornaram-se magos incomparáveis, cada um responsável por uma torre.

Anos depois, o irmão da torre oeste desapareceu com artefatos sagrados, desencadeando uma nova invasão. O rei ordenou a busca dos artefatos e do 4.º irmão.

Trajeto ria do Jogo

A jornada do jogador percorre quatro etapas, desde a invas o inicial de Aerfran at  ao confronto final com o 4.  irm o no Pin culo Malevolente.



Aerfran

Invas o inicial — ponto de partida da aventura.

Bosques Trai oeiros

Primeiro desafio: enfrentar **Pukeko**.

Reino B rbaro

Segunda etapa: confronto com o **Mega B rbaro**.

Pin culo Malevolente

Boss final: o 4.  irm o desaparecido.

Exemplos de Código

O projeto inclui scripts C# para controlo do jogador, armas, inimigos e interface gráfica.

PlayerMotor — Movimento e Salto

```
public class PlayerMotor : MonoBehaviour
{
    private CharacterController controller;
    private Vector3 playerVelocity;
    private bool isGrounded;
    public float speed = 5f;
    public float gravity = -9.8f;
    public float jumpHeight = 3f;

    void Start()
    {
        controller = GetComponent<CharacterController>();
    }

    void Update()
    {
        isGrounded = controller.isGrounded;
    }

    public void ProcessMove(Vector2 input)
    {
        Vector3 moveDirection = Vector3.zero;
        moveDirection.x = input.x;
        moveDirection.z = input.y;

        controller.Move(transform.TransformDirection(moveDir
        ection) * speed * Time.deltaTime);
        playerVelocity.y += gravity * Time.deltaTime;
        if(isGrounded && playerVelocity.y < 0)
        {
            playerVelocity.y = -2f;
            controller.Move(playerVelocity * Time.deltaTime);
        }
    }

    public void Jump()
    {
        if (isGrounded)
        {
            playerVelocity.y = Mathf.Sqrt(jumpHeight * -3.0f *
            gravity);
        }
    }
}
```

PlayerLook — Controlo da Câmera

```
public class PlayerLook : MonoBehaviour
{
    public Camera cam;
    private float xRotation = 0f;
    public float xSensitivity = 30f;
    public float ySensitivity = 30f;

    void Start()
    {
        Cursor.lockState = CursorLockMode.Locked;
    }

    public void ProcessLook(Vector2 input)
    {
        float mouseX = input.x;
        float mouseY = input.y;

        xRotation -= (mouseY * Time.deltaTime) * ySensitivity;
        xRotation = Mathf.Clamp(xRotation, -80f, 80f);

        cam.transform.localRotation =
        Quaternion.Euler(xRotation, 0, 0);
        transform.Rotate(Vector3.up * (mouseX *
        Time.deltaTime) * xSensitivity);
    }
}
```

Mais Código: Arma e Inimigos

Weapon — Disparo e Munição

```
public class Weapon : MonoBehaviour
{
    public bool isActiveWeapon;
    public Camera playerCamera;
    public bool isShooting, readyToShoot;
    public float shootingDelay = 2f;
    public int bulletsPerBurst = 3;
    public int burstBulletsLeft;
    public float spreadIntensity;
    public GameObject bulletPrefab;
    public Transform bulletSpawn;
    public float bulletVelocity = 30;
    public float reloadTime;
    public int magazineSize, bulletsLeft;
    public bool isReloading;

    public enum ShootingMode { Single, Burst, Auto }
    public ShootingMode currentShootingMode;

    private void Awake()
    {
        readyToShoot = true;
        burstBulletsLeft = bulletsPerBurst;
        inputActions = new PlayerInput();
        bulletsLeft = magazineSize;
    }

    private void FireWeapon()
    {
        bulletsLeft--;
        animator.SetTrigger("SHOOTING");

        SoundManager.Instance.shootingSoundfrieren.Play();
        readyToShoot = false;
        Vector3 shootingDirection =
        CalculateDirectionAndSpread().normalized;
        GameObject bullet = Instantiate(bulletPrefab,
        bulletSpawn.position, Quaternion.identity);
        bullet.transform.forward = shootingDirection;
        bullet.GetComponent<Rigidbody>
        ().AddForce(shootingDirection * bulletVelocity,
        ForceMode.Impulse);
        StartCoroutine(DestroyBulletAfterTime(bullet,
        bulletPrefabLifeTime));
    }
}
```

StateMachine — Estados dos Inimigos

```
public class StateMachine : MonoBehaviour
{
    public BaseState activeState;

    public void Initialise()
    {
        ChangeState(new PatrolState());
    }

    void Update()
    {
        if(activeState != null)
        {
            activeState.Perform();
        }
    }

    public void ChangeState(BaseState newState)
    {
        if (activeState != null)
        {
            activeState.Exit();
        }
        activeState = newState;
        if(activeState != null)
        {
            activeState.stateMachine = this;
            activeState.enemy = GetComponent<Enemy>();
            activeState.Enter();
        }
    }
}
```

Conclusão

A realização do projeto **Mage TD** permitiu consolidar e aplicar, de forma prática, os conhecimentos adquiridos ao longo do curso, contribuindo para o desenvolvimento de competências técnicas e metodológicas nas áreas da programação, lógica computacional e motores de jogo.

As dificuldades encontradas foram progressivamente ultrapassadas através de pesquisa, experimentação e testes. Como perspectivas de trabalho futuro, o projeto poderá ser expandido com novos níveis, diversificação de inimigos e aperfeiçoamentos visuais e de desempenho.